



PROGRAMMING FUNDAMENTALS

CS – 101

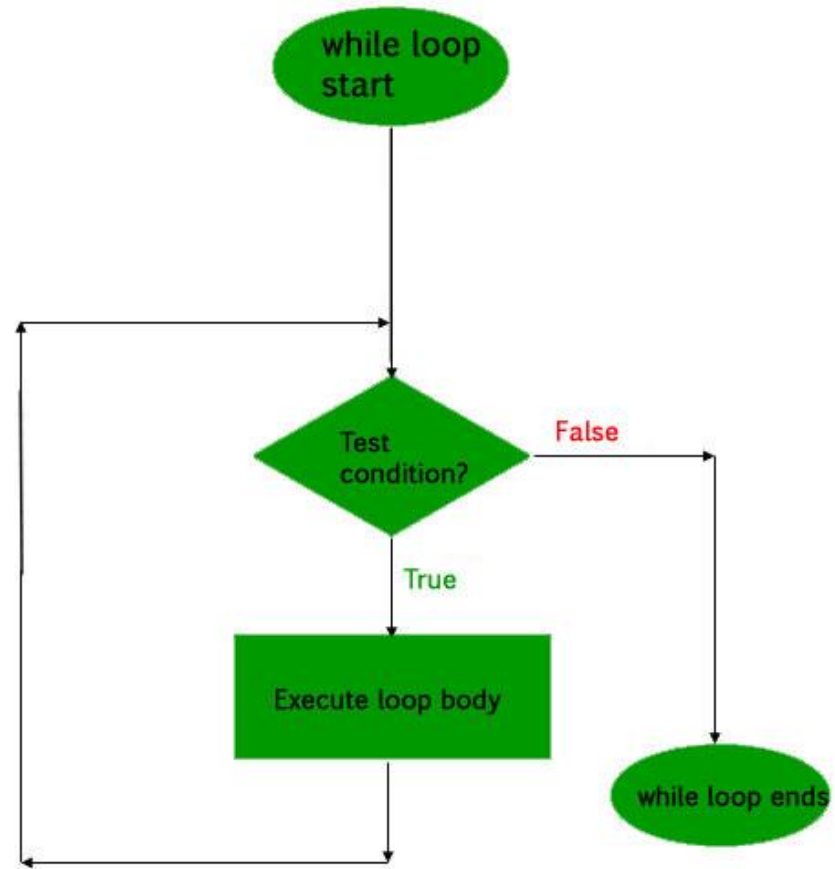
Instructor: Maria N. Samad

October 19th, 2022

WHILE LOOP

- In ***for*** loop, the number of iterations need to be specified in advance
- What happens when we don't know how many times the loop will be executed?
 - Use ***while*** loop
- Loop is terminated on the basis of test condition
- **Syntax:**
 - while expression:
 statement(s)
 statement(s)
- Execute a block of statements repeatedly until a given condition is satisfied

WHILE LOOP



WHILE LOOP

- The three major elements of the loop are also implemented here
- However, the placement of these statements is different
- Executes repeatedly until the test condition is satisfied

EXAMPLES

- `count = 0`
 `while (count < 3):`
 `count = count + 1`
 `print("Hello")`
- The loop runs as long as count is less than 3
- Output?

EXAMPLES

- `n = 10`
 `while (n > 0):`
 `print(n)`
 `n = n - 1`
 `print("Hello")`
- Make sure to include the statements of the loop correctly indented
- Output?

INFINITE LOOP

- When a condition is not placed correctly or updated correctly, the loop may not end
- This makes the loop *infinite* or *endless*
- A common mistake which must always be avoided
- Use correct conditions or correct values to keep the loop finite
- Mostly possible in *while loops* as *for loops* already work for a range of elements

EXAMPLE

- $n = 3$
while ($n \neq 1$):
 print(n)
 if ($n \% 2 == 0$):
 $n = n/2$
 else:
 $n = (n * 2) + 1$
- Output?
- How to correct it?

EXERCISES

1. Write a function that prints octal equivalent value of decimal number, n using loops, where $n \geq 0$
2. Write a function that calculates $\log_2 n$ using while loop, where $n > 1$
3. Write a function that counts the total number of digits in a number, n , using while loops where $n > 0$

EXERCISE # 1 SOLUTION

1. Write a function that prints octal equivalent value of decimal number, n using loops, where $n \geq 0$

- ```
def dec2oct(n):
 octal = ''

 while n > 0:
 octal = str(n % 8) + octal
 n //= 8
 return octal
```
- ```
num = 32  
print(dec2oct(num))
```

EXERCISE # 2 SOLUTION

2. Write a function that calculates $\log_2 n$ using while loop, where $n > 1$

- ```
def logbase2(n):
 count = 0

 while((n//2) > 0):
 count = count + 1
 n = n//2
 return count
```
- ```
num1 = 16  
print(logbase2(num1))
```

EXERCISE # 3 SOLUTION

1. Write a function that counts the total number of digits in a number, n , where $n > 0$
 - ```
def count_digits(n):
 count = 0
 while n != 0:
 n = n // 10
 count = count + 1
 return count
```
  - ```
num1 = 1234567  
print(count_digits(num1))
```

BREAK STATEMENT

- In order to prevent infinite looping, ***break*** statement can be used.
- Lets you jump out of the loop even before the loop is completed
- Can be used for both ***for*** and ***while loops***
- Useful when the prior number of iterations are not known

EXAMPLE (WHILE LOOP)

- Prompt the user to enter “done” in the right case and format, and continue asking until the right input has been entered
- while True:
 line = input()
 if (line == “done”):
 break
 else:
 print(“Wrong input! Try again!”)
print(“Completed”)
- Output?

EXAMPLE (FOR LOOP)

- Search for a letter in a string and stop when the first occurrence of the letter is found
- for letter in "Hello World":
 - if(letter == 'o') or (letter == 'l'):
 - break
 - print("Current letter:", letter)
- Output?

NESTED LOOPS

- Loops that are defined inside another loop
- Any of the iterators/loops discussed in class can be used to define nested loops
 - A ***for*** loop inside another ***for*** loop
 - **Syntax:**
 - for loop_vbl1 in sequence1:
 for loop_vbl2 in sequence2:
 statement(s)
 statement(s)

NESTED LOOPS

- A *while* loop inside a *for* loop

- Syntax:

- for loop_vbl in sequence:
 while expression:
 statement(s)
 statement(s)

- A *for* loop inside a *while* loop

- Syntax:

- while expression:
 for loop_vbl in sequence:
 statement(s)
 statement(s)

NESTED LOOPS

- ***while*** loop inside another ***while*** loop
 - **Syntax:**
 - while expression1:
 while expression2:
 statement(s)
 statement(s)

EXAMPLE

- ```
for i in range(1, 5):
 for j in range(i):
 print(i, end = ' ')
 print()
```
- Output?

# EXERCISES (NESTED LOOPS & BREAK STATEMENT)

1. Write a function `stretch_str` that takes a string argument 's' and prints a new string such that the first character appears once, the second character is repeated twice, the third character is repeated thrice, and so on, **using nested loops**.
2. Write a function `repeated sum` that takes a list as argument, and for every element, it calculates the sum of all the next elements in the list. The function must create a string variable containing all the sums and return it, **using nested loops**. For example:
  - For a given list, `lst1 = [2, 3, 4, 5, 6]`, the return value will be "18, 15, 11, 6" – that is, as a string
3. Print the following pattern **using nested loops** for any value of n:

```
1
2 4
3 5 7
4 6 8 10
5 7 9 11 13
6 8 10 12 14 16
```

4. Write a function `is_float` that checks if the given argument is a floating point number or not. If it is, it returns True, otherwise False
5. Write a function `is_input_correct` that takes input from the user, and without casting it, checks if it is a floating point number or not. If the user has not entered the correct value, it should print an error message: "Invalid Entry! Try again!" and continue prompting the user to enter correctly. Once the correct input is received, then convert to float and then print its type, and end the program

# EXERCISE # 1 SOLUTION

1. Write a function that prints octal equivalent value of decimal number, n using loops, where  $n \geq 0$ 
  - ```
def stretch_str(s):  
    for i in range(len(s)):  
        for j in range(i+1):  
            print(s[i], end = "")
```
 - ```
stretch_str("Gum")
```

# EXERCISE # 2 SOLUTION

2. Write a function `repeated_sum` that takes a list as argument, and for every element, it calculates the sum of all the next elements in the list. The function must create a string variable containing all the sums and return it, using nested loops.

- ```
def repeated_sum(l1):  
    str1 = ''  
    for i in range(1, len(l1)):  
        sum1 = 0  
        for j in range(i, len(l1)):  
            sum1 = sum1 + l1[j]  
        if(i == len(l1) - 1):  
            str1 = str1 + str(sum1)  
        else:  
            str1 = str1 + str(sum1) + ", "  
    return str1
```
- ```
lst1 = [2, 3, 4, 5, 6]
print(repeated_sum(lst1))
```

# EXERCISE # 3 SOLUTION

3. Print the following pattern using nested loops for any value of n

- ```
def pattern(n):  
    for i in range(1,n+1):  
        print(i, end = ' ')  
        count = i  
        for j in range (1,i):  
            count = count + 2  
            print(count, end = ' ')  
        print()  
  
• pattern(6)
```

EXERCISE # 4 SOLUTION

4. Write a function isFloat that checks if the given argument is a floating point number or not. If it is, it returns True, otherwise False

- ```
def isFloat(string):
 count = 0
 dot = 0
 digit_lst = ['0','1','2','3','4','5','6','7','8','9']
 for i in string:
 if (i in digit_lst):
 count = count + 1
 elif (i == "."):
 dot = 1
 else:
 return False
 if(count == (len(string) - 1) and dot == 1):
 return True
 else:
 return False
```
- ```
print(isFloat("3.14"))  
print(isFloat("345"))  
print(isFloat("Hello"))  
print(isFloat("2.3.4"))
```


EXERCISE # 5 SOLUTION

5. Write a function `is_input_correct` that takes input from the user, and without casting it, checks if it is a floating point number or not. If the user has not entered the correct value, it should print an error message: "Invalid Entry! Try again!" and continue prompting the user to enter correctly. Once the correct input is received, then convert to float and then print its type, and end the program

- ```
def isFloat(string): #using the same function for float checking from the last question
 count = 0
 dot = 0
 digit_lst = ['0','1','2','3','4','5','6','7','8','9']
 for i in string:
 if (i in digit_lst):
 count = count + 1
 elif (i == "."):
 dot = 1
 else:
 return False
 if(count == (len(string) - 1) and dot == 1):
 return True
 else:
 return False
```

# EXERCISE # 5 SOLUTION (Cont'd)

- ```
def main():  
    while(True):  
        in1 = input()  
        if (isFloat(in1) == True):  
            i1 = float(in1)  
            print(type(i1))  
            break  
        else:  
            print("Invalid Entry! Try again!")
```
- ```
if __name__ == "__main__":
 main()
```

# SELF PRACTICE TASKS

1. Write a program to find and print all the numbers that are divisible by 7 and are a multiple of 5, between 1500 and 2700 (inclusive)
2. Write a program to prompt the user to guess a number between 1 and 9. User inputs a value. If incorrect, then print message, “Try Again”, and continue asking until correct guess is found. If correct, then print “Well guessed” and end the program
3. Write a Python program that takes a word as input from the user and reverses it, and then prints both the actual word as well as the reversed one {Do NOT use the built in reverse function, instead, use loops to reverse it}

# SELF PRACTICE TASKS

4. Write a program that iterates from 1 to 100. For multiples of 4, it should print “Fizz” instead of that number. For multiples of 7, print “Buzz” instead of the number. For numbers that are multiples of both 4 and 7, print “FizzBuzz”
5. Print the following patterns using Nested Loops

|   |    |    |    |    |    |
|---|----|----|----|----|----|
| 1 |    |    |    |    |    |
| 2 | 4  |    |    |    |    |
| 3 | 6  | 9  |    |    |    |
| 4 | 8  | 12 | 16 |    |    |
| 5 | 10 | 15 | 20 | 25 |    |
| 6 | 12 | 18 | 24 | 30 | 36 |

|   |    |     |     |      |
|---|----|-----|-----|------|
| 5 | 25 | 125 | 625 | 3125 |
| 4 | 16 | 64  | 256 |      |
| 3 | 9  | 27  |     |      |
| 2 | 4  |     |     |      |
| 1 |    |     |     |      |